

QuestLife

Cahier des charges fonctionnel

Application web de gestion d'objectifs personnels gamifiée

Version 2.0 · 17 March 2026 · Usage interne — développement

Document produit par le designer UX/UI — destiné au développeur front-end

Nouveautés v2.0

+ Module 4 — Compte utilisateur (profil gamifié, pseudonyme, série quotidienne, objectifs hebdos)

+ Modèle de données étendu (UserProfile, WeeklyObjective, activityLog)

+ Pool d'objectifs hebdomadaires rotatifs (WEEKLY_OBJECTIVES_POOL)

+ Conformité RGPD : aucune donnée personnelle sensible — stockage local uniquement

Table des matières

- 1 Présentation du projet
- 2 Architecture fonctionnelle
- 3 Module 1 — Chapitres
- 4 Module 2 — Arbre de quêtes
- 5 Module 3 — Récompenses & XP
- 6 Module 4 — Compte utilisateur (NOUVEAU)
 - 6.1 Page Compte — contenu et sections
 - 6.2 Pseudonyme
 - 6.3 Série quotidienne (streak)
 - 6.4 Objectifs hebdomadaires rotatifs
 - 6.5 Conformité RGPD
- 7 Édition de l'arbre
 - 7.1 Création d'une quête (sidebar + drag)
 - 7.2 Modification d'une quête (panel droit)
 - 7.3 Suppression et gestion des dépendances
 - 7.4 Création et suppression de liens
- 8 Navigation entre chapitres
- 9 Création d'un nouveau chapitre
- 10 Flows utilisateur
- 11 Règles métier — tableau récapitulatif
- 12 Spécifications techniques
- 13 Modèle de données
- 14 Priorités de livraison

1 Présentation du projet

Contexte

QuestLife est une application web de gestion d'objectifs personnels inspirée du mod FTB Quests de Minecraft. Elle transpose la mécanique de progression par quêtes — arbre de dépendances visuelles, déblocage progressif, récompenses — dans un contexte de vie quotidienne : santé, carrière, créativité, apprentissage.

Objectif principal

Offrir à l'utilisateur un outil visuel et engageant pour structurer ses objectifs personnels sous forme d'arbres de quêtes interactifs, avec un système de récompenses (XP, niveaux, badges) et un profil gamifié qui valorise la progression dans le temps.

Périmètre v2 (scope)

L'application v2 est 100 % front-end, sans backend ni authentification.

- Application web mono-utilisateur, usage personnel
- Persistance locale via LocalStorage (pas de compte, pas de sync cloud)
- Pas d'application mobile native — responsive non requis en v2
- Pas de collaboration multi-utilisateurs
- Module Compte utilisateur : pseudonyme, XP, badges, série, objectifs hebdos
- Aucune donnée personnelle sensible (conformité RGPD by design)

Stack technique cible

L'application est développée avec React + Vite, React Flow pour le canvas, Tailwind CSS pour le style, et Zustand pour la gestion d'état global.

2 Architecture fonctionnelle

L'application est organisée autour de 4 modules principaux :

Module	Rôle	Composant principal
Chapitres	Conteneurs thématiques (Santé, Carrière...)	Page d'accueil + modal création
Arbre de quêtes	Graphe de dépendances visuelles	Canvas React Flow
Récompenses & XP	Progression, niveaux, badges	Profil + événements
Compte utilisateur	Pseudo, série, objectifs hebdos, stats	ProfilePage (NOUVEAU)

Niveaux de navigation

L'application a 3 niveaux de navigation : page d'accueil (grille chapitres), vue arbre d'un chapitre, et page Compte (profil utilisateur). La page Compte est accessible depuis un bouton icône dans le header de l'accueil.

3 Module 1 — Chapitres

Voir v1.0 — spécifications inchangées.

Les chapitres sont les conteneurs de premier niveau. Chaque chapitre regroupe un arbre de quêtes autour d'un thème de vie. Les fonctionnalités (création, édition, archivage, suppression, réordonnancement) restent identiques à la v1.0.

Propriété	Type	Obligatoire	Description
id	uuid string	Oui	Identifiant unique généré
title	string	Oui	Nom du chapitre
description	string	Non	Description courte
color.bg / color.accent	hex string	Oui	Couleur fond + accent (8 choix)
icon	string (enum)	Oui	Clé icône SVG (16 choix)
order	number	Oui	Position dans la liste
createdAt	timestamp	Oui	Date de création
archivedAt	timestamp	Non	Date d'archivage

4 Module 2 — Arbre de quêtes

Voir v1.0 — spécifications inchangées.

La vue arbre est l'écran central. Elle affiche un canvas interactif avec les quêtes sous forme de nœuds (3 types : standard, habit, boss) reliés par des liens de dépendance. Les statuts (draft, locked, active, done) et la gestion des liens restent identiques.

5 Module 3 — Récompenses & XP

Niveaux

Niveau	XP requis	Titre
1	0	Débutant
2	200	Apprenti
3	500	Explorateur
4	1 000	Aventurier
5	2 000	Expert
6	3 500	Maître
7	5 500	Champion
8	8 000	Légende
9	11 000	Héros
10	15 000	Transcendant

Badges automatiques

Déclencheur	Nom du badge
Première quête complétée	Premier pas
5 quêtes complétées	En route
10 quêtes complétées	Régulier
50 quêtes complétées	Quêteur chevronné
Premier chapitre complété à 100 %	Maître du chapitre
500 XP cumulés	Accumulateur
1 000 XP cumulés	Millionnaire XP
Atteindre le niveau 5	Demi-chemin
Créer 3 chapitres	Organisateur
Compléter une quête de type boss	Boss vaincu

6 Module 4 — Compte utilisateur

NOUVEAU en v2.0

Le module Compte est la page de profil gamifiée de l'utilisateur. Son but est de valoriser la progression dans le temps et de créer des mécaniques d'engagement (série quotidienne, objectifs hebdomadaires, badges visibles) qui donnent envie de revenir chaque jour, comme dans un jeu.

6.1 Page Compte — contenu et sections

Section	Contenu affiché
Hero	Avatar initiales (2 premières lettres du pseudo), pseudonyme éditable, niveau actuel + titre RPG, badges
Stats clés	Quêtes réalisées / total, XP total cumulé, nombre de badges débloqués / disponibles
Activité 7 jours	Graphique en barres (quêtes complétées par jour) + compteur de série quotidienne en cours
Objectifs hebdo	4 défis rotatifs avec bonus XP, progression visible, réinitialisés chaque lundi
Progression chapitres	Barre colorée (accent du chapitre) + compteur x/total pour chaque chapitre
Badges	Grille : badges débloqués en couleur, badges verrouillés grisés (pour créer l'envie de les compléter)

6.2 Pseudonyme

- Saisie libre par l'utilisateur, modifiable à tout moment depuis la page Compte
- Validation : 2 à 20 caractères, lettres, chiffres, _ et - autorisés
- Valeur par défaut à la première ouverture : « Aventurier »
- L'avatar affiche les 2 premières lettres du pseudo en majuscules
- Pas de vrai nom, pas d'email, pas d'identifiant externe — conformité RGPD by design

6.3 Série quotidienne (streak)

La série est un compteur de jours consécutifs durant lesquels l'utilisateur a complété au moins une quête. C'est la mécanique d'engagement la plus puissante — briser une longue série est douloureux et incite à revenir chaque jour.

Condition	Comportement
<code>today == lastActiveDate</code>	Jour déjà compté — ne pas incrémenter
<code>today == lastActiveDate + 1 jour</code>	<code>current++</code> , mise à jour de <code>lastActiveDate</code>
<code>today > lastActiveDate + 1 jour</code>	Série brisée : <code>current = 1</code> , mise à jour de <code>lastActiveDate</code>
<code>longest < current</code>	Mise à jour du record historique (<code>longest</code>)

`updateStreak()` doit être appelé dans `completeQuest()` immédiatement après le crédit XP.

6.4 Objectifs hebdomadaires rotatifs

Chaque semaine (du lundi au dimanche), 4 objectifs sont tirés aléatoirement depuis le pool WEEKLY_OBJECTIVES_POOL. Ils se renouvellent automatiquement — l'utilisateur a toujours un défi frais à relever. Compléter un objectif hebdo crédite des XP bonus.

id	Label	Bonus XP	Cible	Type
w_3quests	Compléter 3 quêtes cette semaine	50	3	questCount
w_5quests	Compléter 5 quêtes cette semaine	100	5	questCount
w_streak3	Maintenir une série de 3 jours	75	3	streak
w_streak5	Maintenir une série de 5 jours	150	5	streak
w_boss	Compléter une quête de type boss	100	1	bossQuest
w_chapter	Compléter un chapitre entier	150	1	chapterDone
w_xp200	Gagner 200 XP cette semaine	80	200	weeklyXP
w_newquest	Créer 2 nouvelles quêtes	30	2	questCreated

- La clé de semaine est au format YYYY-Wnn (ex : 2026-W12)
- Si weeklyObjectives.weekKey ≠ semaine courante → régénérer 4 objectifs au hasard sans doublons
- checkWeeklyObjectives() est appelé après chaque completeQuest()
- Le bonus XP d'un objectif hebdo ne peut être crédité qu'une seule fois (idempotent)

6.5 Conformité RGPD & vie privée

Donnée	Stockée ?	Localisation	Sensible ?
Pseudonyme	Oui	localStorage	Non — choisi librement, aucun lien identitaire
Email	Non	—	N/A
Nom / prénom	Non	—	N/A
XP, badges, quêtes	Oui	localStorage	Non — données de jeu anonymes
Historique activité	Oui (90 j max)	localStorage	Non — compteurs agrégés
Identifiant externe	Non	—	N/A
Données biométriques	Non	—	N/A

Règle de conception : aucune donnée ne quitte l'appareil de l'utilisateur en v2. Tout est stocké exclusivement dans le localStorage du navigateur. Aucun backend, aucune télémétrie, aucun cookie de tracking.

7 Édition de l'arbre de quêtes

Voir v1.0 — spécifications inchangées.

Les sections 7.1 à 7.4 (création par drag, modification via panel droit, suppression avec gestion des dépendances, création/suppression de liens) restent identiques à la v1.0. Se référer au document v1.0 pour le détail.

8 Navigation entre chapitres

Voir v1.0 — spécifications inchangées.

Page d'accueil en grille 2 colonnes, header avec breadcrumb et progression, switcher rapide de chapitres. Ajout en v2 : bouton icône 'Compte' dans le header de la page d'accueil, ouvrant la page ProfilePage.

- Clic icône 'Compte' dans le header → navigation vers ProfilePage
- ProfilePage accessible uniquement depuis l'accueil (pas depuis la vue arbre)

9 Création d'un nouveau chapitre

Voir v1.0 — spécifications inchangées.

La modal de création (nom, description, couleur parmi 8, icône parmi 16) et le sélecteur couleur/icône restent identiques à la v1.0.

10 Flows utilisateur principaux

Compléter une quête

1. Accueil → clic sur un chapitre → vue arbre
2. Clic sur un nœud actif → panel droit s'ouvre
3. Clic sur 'Marquer comme faite' dans le panel
4. Système : +XP au profil, `updateStreak()`, `checkWeeklyObjectives()`, `_checkBadges()`
5. Quêtes dépendantes passent de locked à active
6. Canvas mis à jour, nœud passe en statut done (fond vert)

Consulter son profil

1. Accueil → clic icône 'Compte' dans le header
2. `ProfilePage` s'affiche avec toutes les sections
3. L'utilisateur peut modifier son pseudonyme
4. Les objectifs hebdos affichent la progression en temps réel
5. Les badges verrouillés montrent ce qu'il reste à accomplir

Débloquer un objectif hebdomadaire

1. L'utilisateur complète des quêtes au fil de la semaine
2. `checkWeeklyObjectives()` évalue la progression après chaque complétion
3. Quand `progress >= target` → `done = true`, bonus XP crédité
4. Toast de notification avec le montant du bonus
5. La case objectif passe au vert dans `ProfilePage`

11 Règles métier — tableau récapitulatif

Action	Condition	Comportement attendu	Priorité
Créer nœud	Titre vide	Statut draft, supprimé si panel fermé sans titre	P1
Passer en done	Toujours	+XP profil, updateStreak(), checkWeeklyObjectives(), _checkBadges(), débloc	P1
Ajouter lien	Crée un cycle	Refusé + message erreur	P1
Supprimer quête isolée	Aucun lien	Confirmation simple	P1
Supprimer quête prérequis	A des dépendantes	Modal choix : supprimer liens ou rediriger	P1
Supprimer chapitre	Toujours	Modal + nb quêtes impactées. XP conservés.	P1
Modifier pseudonyme	Validation format	2-20 chars, alphanum + _ - uniquement	P1
Série quotidienne	À chaque completeQuest	Appeler updateStreak() — logique décrite §6.3	P1
Objectifs hebdo	À chaque completeQuest	Appeler checkWeeklyObjectives() — §6.4	P1
Nouveau bonus hebdo	done passe à true	Créditer bonusXP une seule fois (idempotent)	P1
Semaine changée	weekKey ≠ semaine courante	Ré-générer 4 objectifs depuis le pool	P1
Drag nœud	Toujours	Position sauvegardée automatiquement	P2
Badge déclenché	Événement atteint	Toast + badge ajouté au profil	P2
Zoom/pan canvas	Toujours	État mémorisé par chapitre	P2

12 Spécifications techniques

Rôle	Librairie / outil	Version	Justification
Framework UI	React	18+	Écosystème, hooks, performance
Build tool	Vite	5+	DX rapide, HMR, léger
Canvas / graphe	React Flow	11+	Drag, zoom, pan, ports, edges natifs
Style	Tailwind CSS	3+	Utilitaire, consistance
État global	Zustand	4+	Simple, performant, peu de boilerplate
Persistence v2	LocalStorage	Natif	Pas de backend requis
Identifiants	nanoid	—	IDs courts et uniques
Notifications	react-hot-toast	—	Toasts légers pour badges/succès

Nouvelles fonctions store requises (v2)

Fonction	Appelée depuis	Comportement
updatePseudo(pseudo)	ProfilePage	Valide et persiste le pseudonyme
updateStreak()	completeQuest()	Incrémente ou réinitialise la série
getWeeklyObjectives()	ProfilePage, completeQuest()	Retourne ou génère les objectifs courants
checkWeeklyObjectives()	completeQuest()	Met à jour progress/done, crédite bonus XP

13 Modèle de données complet

Structure JSON complète stockée dans LocalStorage (clé : questlife_data) :

Chapter

```
id: string (uuid)
title: string
description?: string
color: { bg: string, accent: string }
icon: ChapterIconKey (enum 16 valeurs)
order: number
createdAt: number (timestamp)
archivedAt?: number (timestamp)
```

Quest

```
id: string (uuid)
chapterId: string
title: string
description?: string
xp: number (default 50)
status: 'draft' | 'locked' | 'active' | 'done'
type: 'standard' | 'habit' | 'boss'
dependencies: string[] (ids des prérequis)
position: { x: number, y: number }
completedAt?: number (timestamp)
```

UserProfile (étendu v2)

```
pseudo: string (défaut 'Aventurier')
totalXP: number
level: number (1-10)
badges: Badge[]
xpHistory: { questId, xp, timestamp }[]
streak: { current: number, longest: number, lastActiveDate: string }
weeklyObjectives: { weekKey: string, objectives: WeeklyObjective[] }
activityLog: { date: string, count: number }[] (90 jours max)
```

WeeklyObjective (NOUVEAU)

```
id: string
label: string
bonusXP: number
target: number
```



```
progress: number
```

```
done: boolean
```

UIState (non persisté)

```
activeChapterId?: string
```

```
selectedQuestId?: string
```

```
isPanelOpen: boolean
```

```
canvasViewport: { [chapterId]: { x, y, zoom } }
```

14 Priorités de livraison

v1 — Fondation	Terminé
• Page d'accueil avec grille de chapitres • Modal création de chapitre (nom, couleur, icône) • Vue arbre React Flow avec nœuds drag-and-drop • Sidebar de création de quêtes (3 types) • Panel éditeur droit (titre, description, XP, statut) • Gestion des liens de dépendance (port drag + menu panel) • Détection et blocage des cycles • Statuts : draft, locked, active, done • Déblocage automatique des quêtes dépendantes • Persistance LocalStorage • Navigation : accueil ↔ arbre, switcher, breadcrumb	
v2 — Gamification + Compte	En cours
• Système XP : attribution à la complétion • Niveaux : 10 paliers avec titres RPG • Badges : 10 déclencheurs automatiques • Toast notifications pour badges débloqués • Profil utilisateur visible dans le header • Animations canvas : edges animés sur quêtes done • MODULE COMPTE : ProfilePage complète • Pseudonyme éditable (validation + avatar initiales) • Série quotidienne (streak) avec historique • Objectifs hebdomadaires rotatifs (pool 8 objectifs) • Graphique activité 7 jours • Conformité RGPD by design	
v3 — Personnalisation	Planifié
• Export JSON (backup complet) • Import JSON (restauration) • Thèmes visuels (clair/sombre/nuit) • Réordonnancement des chapitres par drag • Archivage de chapitres • Statistiques globales (dashboard récapitulatif)	
v4 — Cloud (optionnel)	Futur
• Authentification (email/password ou OAuth)	

- Synchronisation cloud (API REST + base de données)
 - Multi-appareils
 - Partage en lecture seule d'un arbre
-

Note finale au développeur

Ce document constitue la référence fonctionnelle complète pour la version 2.0 de QuestLife. La section 6 (Module Compte) est entièrement nouvelle et constitue la priorité de développement pour cette version. Les sections 7-9 restent inchangées par rapport à la v1.0. Toute ambiguïté doit être remontée avant l'implémentation.